

fichiers en Python

Accès aux fichiers en Python

Alors, dans cette partie de cours, on va découvrir l'Utilisation des fichiers

✓ **En général, pour utiliser un fichier, on a trois étapes fondamentales :**

1. L'ouverture du fichier
2. l'opération voulue (lecture / ecriture/ ajout)
3. La fermeture du fichier (fermer et supprimer)

1. L'ouverture du fichier

- Python a plusieurs fonctions pour créer, lire, mettre à jour et supprimer des fichiers texte.
- La fonction clé pour travailler avec des fichiers en Python est `open()`.

Syntaxe : `f = open(file, mode = 'r')`

- Elle prend deux paramètres; file et mode.
- file : spécifie le nom du fichier à associer au flux;
- mode (facultatif): spécifie le mode ouvert utilisé pour le flux. Le mode d'ouverture par défaut est la lecture en mode texte
- Il existe quatre méthodes (modes) différentes pour ouvrir un fichier:
 - "r" -Lecture -Par défaut. Ouvre un fichier en lecture, erreur si le fichier n'existe pas
 - "a" -Ajouter -Ouvre un fichier à ajouter, crée le fichier s'il n'existe pas
 - "w" -Écrire -Ouvre un fichier pour l'écriture, crée le fichier s'il n'existe pas
 - "x" -Créer -Crée le fichier spécifié, renvoie une erreur si le fichier existe

1. Ouvrir et lire un fichier :

- Pour ouvrir le fichier, utilisez la fonction `open()` intégrée .
 - La fonction `open()` renvoie un objet fichier, qui a une méthode `read()` pour lire le contenu du fichier :
-

```
f = open("demofile.txt","r")  
print(f.read())
```

2. Renvoyez les 5 premiers caractères du fichier :

```
f = open("demofile.txt", "r")  
print(f.read(5))
```

3. Vous pouvez renvoyer une ligne en utilisant la méthode `readline ()`

```
f = open("demofile.txt","r")  
print(f.readline())
```

2. Ecrire dans un fichier

- Pour écrire dans un fichier existant, vous devez ajouter un paramètre à la fonction `open()`:

- Syntaxe(1):

```
f=open("demofile2.txt","a") # ouvrir le fichier en mode ajout (garder l'ancien contenu)
```

```
f.write("Le fichier contient plus de texte") #ajouter la phrase "Le fichier contient plus de texte"
```

- Syntaxe(1):

```
f =open("demofile3.txt","w") #ouvrir le fichier en mode écriture en effaçant son ancien contenu
```

```
f.write("domage!! l'ancien contenu est supprimé") #écrire la phrase domage!! l'ancien contenu est supprimé(mise a jour)
```

3. Fermeture du fichier :

A. Fermer un fichier

- La dernière opération effectuée devrait être la fermeture : **f.close()**. la fonction ne renvoie rien mais ~~lève l'exception en cas d'erreur;~~

```
f=open("demofile.txt","r") #ouvrir le fichier en mode lecture  
print(f.readline()) #lire une line du fichier  
f.close() #fermer le fichier
```

B. Supprimer d'un fichier

- Pour supprimer un fichier, vous devez importer le module OS et exécuter sa fonction **os.remove()**

```
import os #importer la bibliothèque os  
os.remove("demofile.txt") #supprimer un fichier
```

Manipuler les données : Format CSV

- Il existe différents formats standards de stockage de données. Il est recommandé de favoriser ces formats car il existe déjà des modules Python permettant de simplifier leur utilisation.
- Le fichier Comma-separated values (CSV) est un format permettant de stocker des tableaux dans un fichier texte. Chaque ligne est représentée par une ligne de texte et chaque colonne est séparée par un séparateur (virgule, point-virgule ...).
- Les champs texte peuvent également être délimités par des guillemets.
- Lorsqu'un champ contient lui-même des guillemets, ils sont doublés afin de ne pas être considérés comme début ou fin du champ.
- Si un champ contient un signe pouvant être utilisé comme séparateur de colonne (virgule, point-virgule ...) ou comme séparateur de ligne, les guillemets sont donc obligatoires afin que ce signe ne soit pas confondu avec un séparateur.

• Données sous la forme d'un tableau

Nom	Prénom	Age
Dubois	Marie	29
Duval	Julien "Paul"	47
Jacquet	Bernard	51
Martin	Lucie;Clara	14

• Données sous la forme d'un fichier CSV

```
Nom;Prénom;Age
"Dubois";" Marie "; 29
"Dupal";"Julien ""Paul""";47
Jacquet;Bernard;51
Martin;"Lucie;Clara";14
```

1. Lecture d'un fichier CSV

- Pour lire un fichier CSV, il faut ouvrir un flux de lecture de fichier et ouvrir à partir de ce flux un lecteur CSV

```
import csv #importer la bibliothèque csv
fichier = open("exempleCSV.csv", "r")
lecteurCSV = csv.reader(fichier, delimiter=";") # Ouverture du lecteur CSV en lui fournissant le caractère séparateur (ici ";")
for ligne in lecteurCSV:      # parcourir les lignes du fichier CSV
    print(ligne)              # afficher chaque ligne du fichier CSV
fichier.close() #fermer le fichier CSV
```

Affichage:

```
['Nom ', 'Prenom', 'Age']
['Dubois', 'Marie', '29']
['Duval', 'Julien ', '47']
['Jacquet', 'Bernard', '51']
['Martin', 'Lucie', '14']
```

- Il est également possible de lire les données et obtenir un dictionnaire par ligne contenant les données en utilisant DictReader au lieu de reader

```
import csv #importer la bibliothèque csv
fichier = open("noms.csv", "r")
lecteurCSV = csv.DictReader(fichier, delimiter=";")
for ligne in lecteurCSV:
    print(ligne)
fichier.close()
```

Affichage:

```
{'Nom ': 'Dubois', 'Prenom': 'Marie', 'Age': '29'}
{'Nom ': 'Duval', 'Prenom': 'Julien', 'Age': '47'}
{'Nom ': 'Jacquet', 'Prenom': 'Bernard', 'Age': '51'}
{'Nom ': 'Martin', 'Prenom': 'Lucie', 'Age': '14'}
```

2. Écriture dans un fichier CSV :

- À l'instar de la lecture, on ouvre un flux d'écriture (fonction `writer()`) et on ouvre un écrivain CSV à partir de ce flux (fonction `writerow()`) :

```
import csv #importer la bibliothèque csv  
fichier = open("annuaire.csv", "w") #ouvrir un fichier en mode écriture  
ecrivainCSV = csv.writer(fichier, delimiter=";") #ouvre un flux d'écriture  
ecrivainCSV.writerow(["Nom", "Prénom", "Téléphone"]) #écrire une 1ère ligne dans le fichier annuaire.csv  
ecrivainCSV.writerow(["Dubois", "Marie", "0198546372"]) #écrire une 2ème ligne dans le fichier annuaire.csv  
ecrivainCSV.writerow(["Duval", "Julien", "0399741052"]) #écrire une 3ème ligne dans le fichier annuaire.csv  
ecrivainCSV.writerow(["Jacquet", "Bernard", "0200749685"]) #écrire une 4ème ligne dans le fichier annuaire.csv  
ecrivainCSV.writerow(["Martin", "Julie", "0399731590"]) #écrire une 5ème ligne dans le fichier annuaire.csv  
fichier.close() #Fermer le fichier
```

Génération du fichier annuaire.csv

Nom	Prénom	Téléphone
Dubois	Marie	0198546372
Duval	Julien	0399741052
Jacquet	Bernard	0200749685
Martin	Julie	0399731590

- Il est également possible d'écrire le fichier en fournissant un dictionnaire par ligne à condition que chaque dictionnaire possède les mêmes clés.
- La fonction DictWriter() produit les lignes de sortie depuis des dictionnaires . Il faut également fournir la liste des clés des dictionnaires avec l'argument fieldnames

```
import csv #importer la bibliothèque csv
bonCommande = [{"produit": "cahier", "reference": "F452CP", "quantite": 41, "prixUnitaire": 1.6},
                {"produit": "stylo bleu", "reference": "D857BL", "quantite": 18, "prixUnitaire": 0.95},
                {"produit": "stylo noir", "reference": "D857NO", "quantite": 18, "prixUnitaire": 0.95},
                ] #déclarer des dictionnaires
fichier = open("bon-commande.csv", "w") #ouvrir un fichier en écrire
ecrivainCSV = csv.DictWriter(fichier, delimiter=";", fieldnames=bonCommande[0].keys())
#produire des lignes de sortie depuis les dictionnaires,
# fieldnames contient les clés des dictionnaires
ecrivainCSV.writeheader() # Ecrire la ligne d'en-tête avec le titre des colonnes
for ligne in bonCommande: # Parcourir les dictionnaires
    ecrivainCSV.writerow(ligne) #Ecrire une ligne dans le fichier
fichier.close()
```

Génération du fichier bon-commande.csv

```
reference;quantite;produit;prixUnitaire
F452CP;41;cahier;1.6
D857BL;18;stylo bleu;0.95
D857NO;18;stylo noir;0.95
```